

RUSTAIFE

Riccardo Tavolato

05/05/2026

1 Introduzione

RUSTAIFE è un progetto che si propone lo scopo di orchestrare conversazioni testuali tra LLM.

La presente versione usa Ollama come unico provider dei modelli, ma si presta ad implementazioni di ulteriori provider.

2 Premesse allo sviluppo del progetto

Molte delle scelte di percorso intraprese nello sviluppo di RUSTAIFE vedono la loro genesi in una cruciale limitazione sugli strumenti disponibili.

Perché il progetto potesse far uso unicamente di risorse open source, le quali non fossero soggette a limitazioni di utilizzo, è stato escluso in partenza il ricorso a provider che richiedessero il pagamento di token di connessione API.

In conseguenza di ciò nasce la scelta di Ollama.

Anche tale decisione non è priva di controindicazioni: in particolare Ollama non fa uso di servizi di cloud computing, bensì esegue l'intera computazione in locale, il che rende necessario disporre di una macchina in grado di fornire adeguate prestazioni.

Tale requisito rappresenta una limitazione sulla capacità di generare conversazioni particolarmente lunghe, considerata anche la natura incrementale del file di salvataggio.

3 Prima modellizzazione

RUSTAIFE interagisce in prima istanza con l'utente, il quale è tenuto a nominare i modelli da coinvolgere nella discussione, nonché l'argomento e la durata di essa.

Successivamente si occupa della comunicazione API con il provider Ollama per le chiamate ai LLM.

Durante la conversazione crea dapprima e poi sovrascrive continuamente un file testuale che trascrive la conversazione. Separatamente salva un file testuale contenente i dati inseriti inizialmente dall'utente.

4 Design del progetto

Per l'interazione con l'utente si è deciso di avvalersi della command line. A tale scopo è stata programmata CLI (command line interface).

La comunicazione con il provider è stata organizzata tramite step a specificità incrementale, inizialmente definendo gli oggetti necessari per comunicarvi (le struct `ConversationData`, `InquirySettings`, `ResponseSettings`, `Ollama`) e successivamente le procedure di comunicazione (il trait `Provider` con il suo metodo `identify_and_answer`).

Nel lavoro preliminare alla scrittura dell'algoritmo principale sono state anche definite le procedure di acquisizione e salvataggio dati (le funzioni `conversation_mkdir` e `write_and_save`).

L'algoritmo principale si occupa infine dell'imposizione di requisiti di input e dell'orchestrazione della conversazione con chiamate cicliche alle procedure di comunicazione con il provider e a quelle di gestione dati, nonché della formattazione dell'output.

5 Implementazione

5.1 cli.rs

Per lo sviluppo della CLI, il primo modulo creato, è stata utilizzata la crate `clap`, un parser Rust per CLI, che è stato semplicemente derivato come trait per la struct `Interfaccia`. Per la stessa struct sono stati derivati anche i trait `Serialize` e `Deserialize`, contenuti nella crate `serde`, necessari per la traduzione dei dati nel formato JSON.

5.2 save_on_file.rs

Il secondo modulo in ordine cronologico ad essere stato sviluppato è stato `save_on_file`, che contiene le funzioni per la gestione dei dati.

La funzione `conversation_mkdir` sfrutta la crate `chrono` per leggere l'ora corrente e creare, all'interno del progetto, una directory assegnataria del timestamp.

La funzione `write_and_save` legge in input un path, crea le parent directory corrispondenti se non esistono già e scrive la stringa letta in input nel file individuato dal path.

5.3 provider.rs

Il terzo modulo creato è `provider`.

L'enum `ErrorHandling` serve come wrapper dei tipi di errori in cui può incorrere la comunicazione con un provider. In linea generale ci si aspetta che gli errori in cui può incorrere il programma siano durante la connessione HTTP, i quali vengono gestiti con la crate `reqwest`.

Nello stesso modulo si definisce il trait `Provider`, che nel progetto viene implementato solamente per `Ollama`.

5.4 ollama.rs

Il modulo ollama è l'ultimo ad essere stato sviluppato.

Si definisce una struct Ollama (e il relativo costruttore) deputata a contenere l'url del provider e il nome dei LLM che si vogliono usare. Aggiungere un campo dove salvare il client è una scelta implementativa che permette di non doverne creare uno nuovo ad ogni richiesta.

La struct InquirySettings contiene le informazioni da trasmettere tramite la richiesta HTTP e ha una struttura precisa. Infatti l'API di Ollama richiede dei parametri che devono essere corrispondenti a quelli dei campi della struct, così che l'API possa fare un matching corretto una volta deserializzata la struct in un JSON. Un discorso analogo vale per la struct ResponseSettings, che invece contiene il testo della risposta e lo stato della richiesta.

L'implementazione del trait Provider per Ollama definisce la procedura di comunicazione del programma con il provider.

5.5 main.rs

Nel main si crea prima la directory della conversazione corrente usando la funzione conversation_mkdir, poi si esegue il parsing dei dati inseriti dall'utente nella CLI e si salva il relativo file formattato in JSON nella directory creata.

Successivamente i modelli recuperati dall'input vengono istanziati in oggetti di tipo Ollama e salvati in un vettore.

A seconda del numero di LLM e di turni scelti dall'utente, viene inviato un diverso prompt. Nel caso non degeneri di più di un modello e più di un turno il programma itera su modelli e turni aggiungendo ogni intervento in coda al precedente e sovrascrivendo il file della conversazione con quello così ottenuto.

Va notato che per ovviare alla natura stateless di ogni chiamata ad un LLM, è necessario, ad ogni intervento, fornire l'intera discussione pregressa, il che rende l'esecuzione computazionalmente pesante.

L'ultimo turno segue un percorso leggermente differente dai precedenti, in modo che l'ultimo intervento della conversazione sia un sommario della conversazione generato dall'ultimo LLM in lista.